

Techniczny Audyt Bezpieczeństwa Architektury Piaskownic AI: Krytyczna Ocena Założeń Projektowych w Obliczu Wektorów Ataku z 2026 Roku

1. Wprowadzenie i Kontekst Audytu Infrastruktury

Niniejszy dokument stanowi rygorystyczny, techniczny audyt bezpieczeństwa architektury planowanego środowiska izolowanego (piaskownicy) przeznaczonego do wykonywania kodu generowanego przez sztuczną inteligencję oraz obsługi systemów agentowych. Analiza została przeprowadzona przy założeniu ograniczonego zaufania do standardowych, wbudowanych mechanizmów bezpieczeństwa aplikacji AI oraz z uwzględnieniem najnowszych danych o zagrożeniach pochodzących z pierwszego i drugiego kwartału 2026 roku. Krajobraz cyberbezpieczeństwa na przełomie lat 2025 i 2026 wykazuje wyraźne przesunięcie; zagrożenia, które wcześniej klasyfikowano jako teoretyczne ryzyka manipulacji modelem językowym, ewoluowały w narzędzia służące do bezpośredniej kompromitacji infrastruktury obliczeniowej. Jak wskazują raporty OWASP GenAI Security Project za pierwszy kwartał 2026 roku, ataki ukierunkowane na warstwy orkiestracji, tożsamości agentowe oraz łańcuchy dostaw stanowią obecnie krytyczne zagrożenie dla środowisk produkcyjnych.¹

Złożoność systemów opartych na agentach polega na tym, że maszyny te nie tylko przetwarzają tekst, ale również podejmują autonomiczne decyzje i uruchamiają kod bezpośrednio w udostępnionych im środowiskach. Zjawisko to doprowadziło do powstania paradygmatu, w którym prompty użytkowników stają się de facto powłokami systemowymi, co oznacza, że wstrzyknięcie złośliwego kontekstu do modelu może bezpośrednio przekształcić się w zdalne wykonanie kodu na serwerze hosta z pominięciem tradycyjnych zabezpieczeń aplikacyjnych.² Audyt ten poddaje bezkompromisowej ocenie najczęściej przyjmowane założenia architektoniczne, takie jak opieranie izolacji na kontenerach Docker, stosowanie bibliotek wirtualizacyjnych dla języka JavaScript czy implementowanie filtrów sieciowych w przestrzeni użytkownika. Z perspektywy inżynierii bezpieczeństwa każda warstwa planowanego oprogramowania musi zostać zweryfikowana pod kątem odporności na celową i wieloetapową eksploatację ze strony skompromitowanego agenta.

2. Krytyczna Analiza Technologii Izolacji Środowiska Uruchomieniowego

Fundamentem każdej piaskownicy dla agentów sztucznej inteligencji jest technologia izolacji kodu. Analiza wykazuje, że wybór odpowiedniego środowiska wykonawczego determinuje, czy kompromitacja agenta pozostanie incydem w pełni izolowanym, czy też doprowadzi do kaskadowego przejścia całego klastra obliczeniowego oraz eksfiltracji wrażliwych danych korporacyjnych.³

Architektura Oparta na Dockerze i Problem Współdzielonego Jądra Systemowego

Wykorzystywanie standardowych kontenerów platformy Docker jako głównego i jedyne go mechanizmu piaskownicy dla niezaufanego kodu, co często jest domyślnym założeniem w prototypach programistycznych, jest klasyfikowane w 2026 roku jako krytyczny błąd architektoniczny. Kontenery systemu Linux nie stanowią twardej bariery bezpieczeństwa same w sobie, ponieważ opierają się one na współdzieleniu jądra systemu operacyjnego z maszyną hosta za pośrednictwem mechanizmów cgroups (grup kontrolnych) oraz przestrzeni nazw (namespaces).⁵ Wszelkie luki zidentyfikowane w jądrze systemu Linux, a także błędy w samym demonie zarządzającym, mogą zostać natychmiastowo wykorzystane do ucieczki z kontenera i przejęcia kontroli nad węzłem.

Szczególnie destrukcyjnym, lecz nadal powszechnie spotykanym antywzorcem projektowym, jest mapowanie gniazda demona Dockera do wnętrza piaskownicy. Szczegółowa analiza platformy OpenClaw – popularnego asystenta AI – ujawniła katastrofalną podatność sklasyfikowaną jako CVE-2026-27002, gdzie całkowity brak walidacji konfiguracji piaskownicy pozwalał zmanipulowanemu agentowi na uruchomienie kontenera z podmontowanym systemem plików zawierającym ścieżkę do gniazda unixowego demona.⁷ Dostęp do tego interfejsu programistycznego jest strukturalnie równoznaczny z uzyskaniem pełnych uprawnień administratora na maszynie hosta. Złośliwy agent, zainfekowany przez tak zwane pośrednie wstrzyknięcie promptu pochodzące na przykład ze skanowanego repozytorium kodu, może w takiej architekturze wywołać bezpośrednie polecenie uruchamiające kontener alpejski z zamontowanym katalogiem głównym systemu gospodarza, uzyskując bezwarunkowy dostęp do absolutnie wszystkich plików hosta, instalując narzędzia trwałej dominacji oraz bezpowrotnie kompromitując infrastrukturę kluczy publicznych.⁶ Z tego powodu systemy bazujące wyłącznie na standardowych kontenerach nie mogą przetwarzać danych pochodzących z zewnątrz w trybie zautomatyzowanym.⁶

Izolacja w Przestrzeni Użytkownika za Pomocą Architektury gVisor

Rozwiązaniem, które w sposób znaczny redukuje powierzchnię ataku jądra hosta w stosunku do klasycznych kontenerów, jest projekt gVisor. Działa on jako zaawansowany pośrednik implementujący system operacyjny w przestrzeni użytkownika. Przechwytuje on wszystkie wywołania systemowe generowane przez aplikację działającą w piaskownicy i przetwarza je w izolowanym środowisku, drastycznie ograniczając liczbę bezpośrednich i potencjalnie niebezpiecznych wywołań przekazywanych do rzeczywistego jądra systemu operacyjnego hosta.¹¹

Pomimo ewidentnego wzrostu bezpieczeństwa architektonicznego, wdrożenie gVisora wykazuje specyficzne ograniczenia, które muszą zostać uwzględnione w planowaniu piaskownicy. Przechwytywanie wywołań systemowych na poziomie przestrzeni użytkownika wprowadza nieunikniony narzut wydajnościowy rzędu od kilkunastu do nawet trzydziestu procent dla operacji intensywnie korzystających z operacji wejścia i wyjścia.¹¹ Dodatkowo, kompromitacja samego gVisora zazwyczaj sprowadza się do ucieczki na poziomie procesu, a

nie pełnej wirtualizacji maszynowej.¹⁴ Z perspektywy wektorów ataku z 2026 roku, istotne jest prawidłowe zarządzanie uprawnieniami samego środowiska uruchomieniowego gVisor. Badacze bezpieczeństwa zauważają, że błędy w konfiguracji środowiska uruchomieniowego, takie jak nieautoryzowane nadanie flagi sieciowej hosta podczas powoływania instancji, mogą zniweczyć jego architektoniczne zalety, co w modelu zagrożeń jest klasyfikowane jako błąd warstwy orkiestracji, a nie samej piaskownicy jako takiej.¹⁵ Niemniej jednak, gVisor skutecznie mityguje wiele krytycznych luk jądra Linux; na przykład w przypadku systemów zarządzanych wykorzystujących tę technologię udowodniono skuteczność blokowania eksploatacji luki CVE-2026-31431, która w innych warunkach pozwalała na nadpisywanie pamięci podręcznej stron systemowych i eskalację uprawnień.¹⁷

Wirtualizacja Sprzętowa jako Standard dla Systemów Wielodostępnych

Złotym standardem w architekturze piaskownic przeznaczonych dla środowisk wielodostępnych oraz wykonujących całkowicie obcy kod generowany dynamicznie przez sztuczną inteligencję są aktualnie lekkie maszyny wirtualne, określane mianem MicroVMs. Do najpopularniejszych technologii z tej kategorii należą AWS Firecracker oraz Kata Containers, które w odróżnieniu od izolacji na poziomie oprogramowania, wykorzystują wirtualizację sprzętową zapewniając najsilniejszą możliwą barierę.¹⁴

Poniższe zestawienie systematyzuje krytyczne różnice między dostępnymi mechanizmami izolacyjnymi pod kątem ich przydatności do budowy infrastruktury agentowej:

Technologia Izolacji	Podstawowy Mechanizm Bezpieczeństwa	Czas uruchomienia (Cold start)	Narzut wydajnościowy (Overhead)	Profil Podatności i Architektoniczne Słabości
Docker (Standard)	Namespaces, cgroups, ograniczenia seccomp	~90ms - do kilku sekund	Minimalny (bliski natywnemu)	Współdzielone jądro hosta; absolutny brak wbudowanej izolacji sprzętowej; ucieczki uwarunkowane lukami kernela. ⁵
gVisor (runsc)	Jądro w przestrzeni użytkownika	50-100ms	10-30% dla operacji I/O	Znaczne obciążenie procesora przy

	(syscall proxy)			dużej liczbie wywołań systemowych; potencjalne obejścia limitów procesowych. ¹¹
Firecracker / Kata	Wirtualizacja sprzętowa (KVM MicroVM)	100-300ms	2-15% w zależności od hipernadzorcy	Zwiększona złożoność infrastrukturalna; trudniejsza integracja z narzędziami ciągłego wdrażania. ¹¹
WebAssembly (WASM)	Izolowana maszyna wirtualna typu stosowego	Poniżej 50ms	Bardzo niski	Ekstremalne restrykcje dostępowe; brak wsparcia dla podprocesów systemowych i zaawansowanych bibliotek natywnych. ⁴

Alternatywa Aplikacyjna: Piaskownice WebAssembly

Nowatorskim i wysoce bezpiecznym podejściem, obserwowanym w zaawansowanych systemach z 2026 roku, jest realizacja piaskownic wewnątrz kontenerów bezserwerowych za pomocą technologii WebAssembly, co często osiągnięte jest poprzez kompilację środowiska języka Python do postaci WASM.⁴ Architektura ta de facto uniemożliwia bezpośredni dostęp do warstwy sieciowej i fizycznego dysku hosta z założenia samego projektu, co zapewnia absolutną izolację bez konieczności utrzymywania złożonych wirtualizatorów systemu operacyjnego.⁴ Osiągnięto w ten sposób paradygmat, w którym kod agenta posiada dostęp wyłącznie do przydzielonego bufora pamięci i ściśle określonych interfejsów importu danych. Podejście to doskonale sprawdza się do szybkiej izolacji skryptów analitycznych tworzonych w sposób ciągły przez model językowy, jednak posiada bardzo restrykcyjne ograniczenia, polegające między innymi na obsłudze wyłącznie prekompilowanych bibliotek matematycznych oraz całkowitym braku możliwości uruchamiania złożonych procesów podrzędnych systemu, z których agenci orkiestracyjne często muszą korzystać w ramach

testowania oprogramowania.⁴ Z tego powodu WebAssembly nie może stanowić wyłącznej podstawy środowiska agentowego o charakterze ogólnym.

3. Audyt Mechanizmów Wykonawczych i Systemowych Wektorów Ucieczki (Sandbox Escapes)

Analiza techniczna infrastruktury przeznaczonej dla sztucznej inteligencji musi opierać się na pesymistycznym założeniu, iż aplikacyjna piaskownica kontrolująca bezpośrednio wykonanie kodu prędzej czy później zostanie przełamana. Dowodem na konieczność stosowania strategii obrony w głąb (defense in depth) są bezprecedensowe fale krytycznych podatności zaobserwowane w bibliotekach ewaluacyjnych i środowiskach uruchomieniowych na przełomie 2025 i 2026 roku. Poważnym zagrożeniem jest projektowanie rozwiązań, w których granica bezpieczeństwa zaczyna się i kończy na poziomie walidacji kodu przed jego wykonaniem.

Katastrofalne Załamanie Architektury Izolacyjnej Biblioteki vm2

Dla biblioteki vm2, będącej niezwykle popularnym środowiskiem służącym do rzekomo bezpiecznego izolowania i wykonywania niezaufanego kodu JavaScript w aplikacjach Node.js, początek maja 2026 roku okazał się okresem całkowitego i ostatecznego strukturalnego załamania architektury bezpieczeństwa. W niezwykle krótkim czasie opublikowano falę trzynastu nowo odkrytych, krytycznych podatności, pozwalających na pełną ucieczkę z aplikacyjnej piaskownicy i wykonanie arbitralnego kodu na serwerze.² Luki te, charakteryzujące się najwyższymi możliwymi ocenami dotkliwości (CVSS od 9.0 do 10.0), podważają sens stosowania filtracji kodu w warstwie oprogramowania Node.js.

Architektura samej biblioteki vm2 opierała się od dawna na ciągłym łataniu i implementowaniu czarnych list dla specyficznych metod w języku JavaScript, co ostatecznie okazało się strategią niemożliwą do obrony przed zaawansowanymi badaczami bezpieczeństwa i ewoluującymi modelami AI. Do najbardziej wyrafinowanych i brzemiennych w skutki błędów z tej serii należy luka oznaczona jako CVE-2026-22709, która umożliwiła ominięcie mechanizmów czyszczenia wywołań zwrotnych w asynchronicznych strukturach obietnic (Promise callback sanitization bypass).² Umożliwiło to kodowi wykonującemu się wewnątrz zamkniętej strefy na uzyskanie niezdezynfekowanego odniesienia do obiektu macierzystego z zewnątrz piaskownicy. Kolejne zidentyfikowane luki ujawniły niszczycielski potencjał manipulacji prototypami obiektów; zanieczyszczanie prototypów w spójnym połączeniu ze wstrzykiwaniem niebezpiecznych dyrektyw do zapytań o właściwości (CVE-2026-44005 oraz CVE-2026-44006) pozwoliło na wykreowanie ścieżki prowadzącej bezpośrednio z ograniczonego środowiska izolowanego prosto do natywnych procesów węzła.²

Mechanizm działania tych wieloetapowych ataków demonstruje, dlaczego poleganie na vm2 jest niedopuszczalne w architekturze piaskownicy AI. W pierwszym kroku niezaufany skrypt, wygenerowany przez poddany uprzednio wstrzyknięciu model LLM, trafia do biblioteki. Następnie, wykorzystując wyżej wspomniane wektory manipulacji strukturą obietnic lub wymuszaniem wyjątków, przekracza on rzekomo hermetyczną barierę wykonawczą, zyskując potajemny dostęp do macierzystych interfejsów API wbudowanych w Node.js.² Ostatni etap

polega na przejęciu pełnej kontroli nad serwerem; atakujący omija naiwne białe listy, uzyskując bezwarunkowy dostęp do przestrzeni systemu plików, mechanizmów sieciowych czy w końcu do modułu generującego podprocesy systemowe (co realizuje luka CVE-2026-43999), co ostatecznie prowadzi do w pełni interaktywnej powłoki zwrotnej.² Z punktu widzenia architektonicznego, środowiska bazujące na języku JavaScript nie mogą używać vm2 jako granicy bezpieczeństwa, lecz wyłącznie jako pomocniczego narzędzia do strukturyzacji kodu, pod którym musi bezwzględnie znajdować się silna izolacja systemu operacyjnego. Poniższa tabela przedstawia wybrane, najistotniejsze z punktu widzenia projektowego, luki z fali maja 2026 roku kompromitujące izolację poziomu aplikacji:

Identyfikator Podatności	Naruszony Wektor Architektoniczny (Technika)	Potencjalny Skutek dla Infrastruktury
CVE-2026-22709	Ominięcie sanitacji dla wywołań zwrotnych w obietnicach asynchronicznych.	Przedostanie się referencji macierzystych do wnętrza piaskownicy. ²
CVE-2026-26956	Luki w ewaluacji znaczników kodu i modułów skompilowanych WASM.	Bezpośrednie zdalne wykonanie kodu na serwerze matierzystym. ²
CVE-2026-43999	Skuteczne ominięcie mechanizmów białych list dla API systemowego.	Uruchomienie modułu child_process powodujące otwarcie powłoki. ²
CVE-2026-44005	Skażenie struktury prototypów obiektowych (Prototype pollution).	Eskalacja uprawnień wewnętrznych z racji przełamania granic obiektów. ²

Systemowe Podatności Środowiska Uruchomieniowego Runc

Skupiając się na środowisku kontenerowym niezbędnym do orkiestracji agentów sztucznej inteligencji, należy dogłębnie przeanalizować defekty ujawnione w środowisku runc, będącym głównym komponentem odpowiedzialnym za powoływanie kontenerów m.in. dla systemów Docker. Trzy fundamentalne błędy opublikowane pod koniec 2025 roku obnażyły kruchość izolacji w systemie Linux w kontekście współdzielonych mechanizmów montowania zasobów systemowych.¹⁹

Pierwszym z nich, CVE-2025-31133, jest defekt koncepcyjny w zabezpieczeniu znanym jako `maskedPaths`, którego celem jest ochrona kluczowych dla jądra hosta ścieżek dostępowych poprzez maskowanie ich specjalnym węzłem `/dev/null`. Z racji rażącego braku odpowiedniej walidacji operacji na węzłach przez oprogramowanie `runc`, agent dysponujący wstępnym dostępem do generowania środowiska mógł podstępnie zastąpić lokalny zasób `/dev/null` symbolicznym dowiązaniem prowadzącym prosto do bezcennych i wrażliwych lokalizacji pamięci jądra, takich jak `/proc/sys/kernel/core_pattern`. Gdy proces inicjalizacyjny próbował wykonać rutynowe maskowanie, w rzeczywistości nieświadomie otwierał napastnikowi natywny interfejs zapisu do pamięci kernela, co skutkowało błyskawicznym zdemontowaniem wszelkich barier kontenerowych i uzyskaniem pełnej dominacji nad serwerem bazowym.²¹ Kolejna ujawniona luka, sklasyfikowana jako CVE-2025-52565, opierała się na klasycznym zjawisku wyścigu zdarzeń (*race condition*) występującym w newralgicznym momencie inicjalizacji środowiska wewnątrz samego kontenera. Błąd ten polegał na możliwości wyprzedzenia mechanizmów walidacyjnych poprzez celową modyfikację wirtualnego interfejsu terminala (`/dev/pts/$n`) i przekierowanie go, za pomocą odpowiedniego dowiązania, na terminal hosta `/dev/console`. Zmiana ta była celowo planowana i egzekwowana z dokładnością do ułamka sekundy przed globalną aktywacją restrykcji odczytu chroniących pamięć operacyjną, wymuszając w ten sposób na uprzywilejowanym demonie `runc` nieświadome podmontowanie groźnego i zmanipulowanego dowiązania. Zapewniało to napastnikowi bezprecedensowy dostęp polegający na nieograniczonych uprawnieniach modyfikacji plików zarządzania energią, buforami kernela czy wyzwalania akcji paniki całego systemu operacyjnego maszyny gospodarza.¹⁹

Uzupełnieniem tego wektora ataku był błąd CVE-2025-52881, który demonstrował zaawansowaną technikę przekierowywania operacji zapisu dokonywanych w obrębie interfejsu `sysctl`, wykorzystując specyficzne zachowanie współdzielonych punktów montowania (*shared mounts*) w systemie operacyjnym Linux. Napastnicy potrafili bezbłędnie ominąć etykietowanie nakładane przez Moduły Bezpieczeństwa Jądra Linux (LSM relabeling), wprowadzając celowo dezinformację wymuszającą na `runc` realizowanie krytycznych zapisów parametrycznych bezpośrednio na chronione i zastrzeżone obiekty, w tym na potężny przełącznik zachowania systemu `/proc/sysrq-trigger`, co ostatecznie umożliwiało nie tylko ucieczkę z kontenerowej piaskownicy, ale i absolutne przejęcie zarządzania zasobami procesora maszyny sprzętowej hosta.¹⁹

4. Obejścia Autoryzacji Demona Docker i Eksploatacja Protokołu HTTP

Naiwność wielu planów architektonicznych z zakresu wdrażania sztucznej inteligencji polega na założeniu, że systemy decyzyjne oparte na wtyczkach i politykach pośredniczących obronią infrastrukturę przed wygenerowaniem uprzywilejowanego środowiska przez agenta posiadającego ograniczony dostęp do interfejsu programistycznego demona. Dowodem na błędność tej tezy jest wykryty przez zespół badawczy Cyera krytyczny defekt w komponencie autoryzacji samego systemu Docker, oznaczony jako CVE-2026-34040, który ujawnił niepokojącą niekompetencję w procesowaniu żądań sieciowych HTTP.²⁵

Podatność ta, oszacowana na 8.8 w skali CVSS, obnażyła słabości w mechanizmach zabezpieczeń API Dockera, opierających się na pluginach autoryzacyjnych takich jak Open Policy Agent, używanych powszechnie do egzekwowania zasad takich jak zakaz montowania nadrzędnego systemu plików czy limitowanie tworzenia uprzywilejowanych środowisk. Problem polegał na sposobie filtrowania danych wejściowych w komponencie middleware. Zgodnie z analizą kodu w języku Go, proces weryfikacyjny określał konieczność sprawdzenia zawartości ładunku przy pomocy następującej koniunkcji warunków logicznych: konieczność przesłania ładunku, stwierdzenie czy żądanie faktycznie posiada ładunek lub korzysta z transferu porcjowanego, oraz weryfikację, czy zadeklarowana wielkość ładunku jest mniejsza od ustalonej z góry wielkości maksymalnej wynoszącej równo jeden megabajt.²⁵

Błąd omijający te zasady okazał się banalnie wręcz prosty dla każdego procesu lub zautomatyzowanego agenta zdolnego do wystosowania zapytania sieciowego: wysłanie ładunku formatu JSON, którego struktura została intencjonalnie spuchnięta i wypełniona nieużytecznymi spacjami lub sztucznymi danymi do rozmiaru przekraczającego barierę 1MB, powodowało ewaluację ostatniego warunku ograniczenia jako absolutnie fałszywą. Konsekwencją tego błędu projektowego w bramie decyzyjnej był fakt, iż warstwa pośrednicząca dyskwalifikowała ten nadmiarowy ładunek z procesu analizy bezpieczeństwa i przesyłała w jego miejsce pustą deklarację do wtyczki autoryzacyjnej. Wtyczka autoryzacyjna, analizując puste żądanie, nie identyfikowała żadnych złośliwych elementów (takich jak instrukcje kreacji powłoki uprzywilejowanej), tym samym akceptując całkowicie proces. Po uzyskaniu wstępnej autoryzacji demon Dockera przejmował kontrolę i przystępował do realizacji zapytania, opierając się na oryginalnym, przetadowanym żądaniu sieciowym przesyłanym wprost z buforów, i wykonywał je z wszystkimi niszczycielskimi instrukcjami, tworząc kontener zdolny do zamontowania głównego systemu plików oraz kradzieży bezcennych kluczy dostępowych administratora.²⁵

Co niezmiernie istotne, w przypadku wyrafinowanego wariantu stosującego przesyłanie strumieniowe bez deklaracji wagi całkowitej (Chunked Transfer Encoding), zmienna odpowiedzialna za wagę przyjmowała programistyczną wartość minus jeden. Pierwszy etap weryfikacyjny akceptował to podanie z racji rzekomo braku naruszenia limitu, przekazując sterowanie funkcji czytającej porcje. Funkcja ta ulegała jednak cichemu wycofaniu i zwracała brak zawartości w momencie gdy napotykała nieprzerwany i przekraczający założony próg bezpieczeństwa potok informacji, co prowadziło to tożsamego wyniku akceptacji pustej weryfikacji i wykonania złośliwych procedur bezpośrednio w demonie jądrowym Dockera.²⁵ Luka ta była bezpośrednią konsekwencją powierzchownych i niekompletnych napraw wdrożonych podczas łatania bardzo podobnego błędu o symbolu CVE-2024-41110, gdzie inżynierowie zabezpieczyli jedynie wektory posługujące się zerową wagą ładunku sieciowego, pozostawiając górną granicę kompletnie niezabezpieczoną i ignorowaną.

5. Złożona Krytyka Zabezpieczeń Sieciowych i

Problem Filtrowania Egress

Skuteczne i bezkompromisowe środowisko piaskownicy sztucznej inteligencji, zgodne z założeniami z 2026 roku, wymaga zastosowania radykalnej i bezwzględnej polityki izolacji i

kontroli wyjściowego ruchu sieciowego (egress filtering). Zabezpieczenia te muszą prewencyjnie powstrzymywać agentów zarówno przed dokonywaniem szkodliwej eksfiltracji danych korporacyjnych poprzez ukryte kanały na zewnątrz organizacji, jak i przed realizowaniem skanowania i inwentaryzacji otwartych portów w ramach poufnej sieci wewnętrznej. Wybór narzędzi kontrolnych jest w tym przypadku aspektem decydującym, a błędy w ich konfiguracji dają złudne poczucie bezpieczeństwa.

Niewystarczalność Klasycznych Zapor Węzłowych i Obejścia Infrastruktury DNS

Rozwiązania oparte na konwencjonalnych pakietach narzędzi kontrolnych i powszechnie stosowanym oprogramowaniu na maszynach zajmujących się ciągłą integracją czy ewaluacją kodu z reguły ograniczają swoje działania naprawcze do sprawdzania nagłówków IP, definiowania dozwolonych pul adresowych oraz naiwnego śledzenia nieszyfrowanego ruchu sieciowego generowanego przez protokół UDP na porcie domyślnym usługi DNS.³ Architektura taka charakteryzuje się fatalną podatnością na tunelowanie danych, czego doskonałym przykładem są techniki eksfiltracyjne opierające się na modyfikacji żądań serwerów nazw w celu przenoszenia zakodowanych ładunków informacyjnych do autorytatywnych stref napastnika.

Agenty sztucznej inteligencji, napędzane wrogim promptem operacyjnym, nagminnie nadużywają luk uwidoczonych w popularnych instalacjach środowisk deweloperskich. Błędy koncepcyjne zapor sieciowych uwypuklają się przy próbach izolacji zapytań DNS transportowanych wewnątrz warstwy TCP, co pozwala agentowi na wygenerowanie specjalnego polecenia powłoki narzucającego wymuszony transport pożądanego żądania za pośrednictwem połączenia zorientowanego na połączenie do serwera dostawcy chmurowego, wymykając się standardowym filtrom protokołów bezpołączeniowych i omijając w ten sposób blokady.²⁹ Jeszcze bardziej skutecznym mechanizmem stosowanym na szeroką skalę jest wykorzystywanie mechanizmów zapytania DNS rezydującego we wnętrzu bezpiecznego i poprawnie skonfigurowanego tunelu HTTPS (tzw. technologia DoH - DNS over HTTPS). Proces zapory, napotykając silnie zaszyfrowany, oficjalny i uznany za poprawny ruch sieciowy nawiązywany w kierunku korporacyjnych systemów resolverów publicznych, podejmuje decyzję o jego zignorowaniu pod kątem zawartości.²⁹ Uniemożliwia to detekcję procederu, w którym potężnie rozbudowane ładunki informacji ukryte w strukturze zapytań zostają nieskrępowanie przesłane do domeny docelowej przestępcy, wyprowadzając tajne zmienne operacyjne infrastruktury hosta.³

Podatności Nowoczesnych Architektur Bazujących na eBPF (Slicing Attack)

W najnowocześniejszych podejściach do architektury środowisk maszyn wirtualnych coraz powszechniej stosuje się zapory o imponującej wydajności implementowane jako wysoce innowacyjne skrypty rezydujące tuż przy jądrze w postaci podprogramów eBPF (Extended Berkeley Packet Filter). Stanowią one o wiele wyższy stopień kontroli, podejmując analizę zapytań sygnaturowych w locie już na poziomie operacji sprzętowych jądra, co niweluje

problem obciążeniowy systemu gospodarza.³⁰ Niestety, analizy wydajności udowodniły, że rozwiązania te nader często padają ofiarą zjawiska architektonicznej bezstanowości. Metoda, określana jako "Slicing Attack", opiera się na celowej i matematycznie zaprogramowanej fragmentacji strumieni transferowych na warstwie protokołów transmisyjnych. Działanie to powoduje dekompozycję wyszukiwanych i wrogich łańcuchów znakowych definiujących na przykład klucze prywatne w strukturach wieloportowych.³² W efekcie mechanizm eBPF ocenia fragmentarycznie podane dane pakietu, analizując poszczególne zbitki danych niezależnie od ich całokształtu i akceptując ich transferowanie na zewnątrz z racji niezgodności cząstkowych i wybrakowanych fragmentów względem wzorcowych i predefiniowanych list zakazanych pojęć i zwrotów, niwecząc cały sens wdrożenia takiej inspekcji sprzętowej.³²

Defekty Izolacji Węzłów Klastrowych i Problemy Routing'u

Ogromne niebezpieczeństwo płynie również z wdrożenia piaskownic bazujących na systemach platformy Docker w architekturze roju kontenerowego (swarm). Typowym rozwiązaniem mającym uniemożliwić kontenerom wyjście na zewnątrz jest wdrożenie zmiennych systemowych oznaczających zaufane proxy oraz ustanawianie zmiennych środowiskowych nakazujących wykorzystywanie tego wektora. Powszechnie implementowanym w 2026 roku błędem projektowym jest aplikowanie list autoryzacji adresowych (zmienna pomijająca NO_PROXY) z intencją polepszenia latencji oraz zmniejszenia opóźnień komunikacji bazodanowej i usług mikrokontenerowych.³⁴ Agent poddany eksploatacji błyskawicznie analizuje te otwarte luki w regułach autoryzacji sieciowych dla usług wewnętrznych klastra i bez trudu dociera na porty nienadzorowane przez ochronne systemy zaporowe. Równocześnie, implementacja domyślnych interfejsów routingowych w połączeniu ze starszymi edycjami narzędzi systemowych platform dokeryzacyjnych (powszechnie występujących na serwerach deweloperskich i środowisk testowych) skutkuje nadpisywaniem głównych tablic reguł przekierowania jądra linuxa. To, co planowano ograniczyć, nieoczekiwanie obala reguły domyślne jądra gospodarza i wymusza drastyczne i natychmiastowe otwarcie bezpośrednich, podatnych bram sieciowych prowadzących bezpośrednio do podmontowanych interfejsów usług i piaskownic testowych, naruszając izolację bez powiadamiania podsystemów.²⁸

6. Ochrona Zrównoważona Przed Destabilizacją: Wyczerpanie Zasobów i Problemy Narzutu Pamięciowego

Integralnym aspektem stabilności architektonicznej, notorycznie lekceważonym podczas analizy piaskownic generatywnych sztucznych inteligencji, jest odpowiednie zabezpieczenie węzłów obliczeniowych przed brutalnymi atakami wyczerpywania zasobów sprzętowych (Denial of Service). Zważywszy na nieprzewidywalną z definicji naturę i strukturę logiki tworzonej przez autonomiczne modele, tworzenie przez nie skrajnie nieefektywnych skryptów zapętlonych w iteracyjnej nieskończoności to zdarzenia wysoce powszechne.

Pamięć Wirtualna Środowiska Tymczasowego i Ciche Limity OOM

Mechanizmy ograniczania alokacji zasobów za pomocą wskaźników aplikacyjnych (jak na przykład użycie flag ograniczających rozmiar przydzielanej pamięci podczas procedury rozruchowej kontenera) nierzadko są iluzoryczne. Rozpatrując logikę działania kontenerów i analizę alokacji wewnątrz grup kontrolnych na poziomie jądra sprzętowego, środowisko musi zdawać sobie sprawę z tego, że skrypty w środowisku wykonawczym pochłaniają przestrzeń w obszarach strukturalnych systemu plików hosta, a nie samej sterty obliczeniowej procesu wywołującego narzędzie (np. Heap w maszynach interpretujących język, który zajmuje ułamek całokształtu zgłoszonego żądania).³⁶

Krytycznym problemem w przypadku wirtualizacji bezserwerowych do analizy danych wejściowych jest częste deklarowanie punktów montowania typowych stref tymczasowych tmpfs w celu rzekomego przyspieszenia odczytu z wirtualnych ramdysków. Należy jednoznacznie podkreślić, że każda dana transferowana na ten system plików jest automatycznie i nieuchronnie rozliczana względem puli limitów pamięci zarezerwowanej dla struktury węzła maszyny wirtualnej.³⁸ Agent sztucznej inteligencji, nie zważając na procedury usuwania pozostałości obliczeniowych, nagminnie przetwarza przeogromne obiekty danych plikowych, ładując je do wyznaczonych partycji, co stwarza warunki dla lawinowego pożerania zadeklarowanej puli i nagłego załamania usług, podczas gdy sam system czyszczenia struktur operacyjnych nie wykrywa anomalii, albowiem fizyczne wartości podlegają strukturalnie zewnętrznemu systemowi wirtualizującemu. Następstwem tego stanu jest bezzwłoczne wkroczenie jądrowego mechanizmu OOM-killer (Out Of Memory Exception), który bezpardonowo przerywa cykl bytu najważniejszych obiektów struktury podrzędnej.³⁶ Rekomenduje się tu absolutne i obligatoryjne wprowadzanie dedykowanych rozmiarów objętościowych w deklaracjach punktów plików, precyzyjnie definiujących narzucone marginesy dopuszczalności we wpisach montażowych systemów piaskownicy.³⁸

Eksplzja Tabel i Wirtualne Bomby Forkowe

Zjawiskiem niezwykle niebezpiecznym wynikającym bezpośrednio z oddania kontroli generowania kodu zaufanej warstwie agentowej jest generowanie struktur opisywanych jako potoczna bomba forkowa. Eksperymenty polegające na umożliwieniu testowania powłoki skryptami pisanymi przez modele zaowocowały powstawaniem kodu, który powołuje kolejne węzły podrzędne zjawiskiem lawinowego mnożenia poprzez wielokrotne duplikowanie samych siebie przy użyciu procedury instrukcji rekursywnych (wystarczy nieskończona pętla z użyciem wywołania systemowego klonowania własnej istoty w systemie Linux w wywołaniu Pythona).⁴¹ Jeżeli proces wykonawczy piaskownicy operacyjnej oparty będzie na środowisku dockerowym bez nałożonej i starannie oszacowanej bariery numerycznej limitującej zasoby procesowe podprogramów, system wejdzie natychmiastowo w stan zaprzestania przyjmowania zadań w warstwie globalnej jądra Linuxa ze względu na ostateczne sczerpanie dostępnej puli rezerwowanej dla operacji (kernel.pid_max).⁴² Sytuacja ta powoduje wyłączenie całego systemu dla wszystkich pozostałych procesów chmury korporacyjnej czy innych zrównoległonych sesji wielodostępnych, stanowiąc destrukcyjny błąd projektowy. Ochrona w

postaci parametru twardego sufitu procesów (takiego jak flaga --pids-limit) to element, którego nie może zabraknąć w autoryzowanym uruchomieniu skryptowym dla każdego pojedynczego incydentu wykonawczego.⁴⁴

Poniższa tabela stanowi obligatoryjne zestawienie konfiguracyjne parametrów obciążeniowych Dockera w zastosowaniach AI:

Flaga Środowiska Uruchomieniowego	Architektoniczna Funkcja i Potencjał Obronny	Realne Następstwo Naruszenia Zasad dla Hosta
--memory=Xm	Twarde odcięcie rezerwacji całościowej dla cgroup.	Interwencja demona jądra (OOM-killer) zabijającego agresywny skrypt zanim uderzy w sprzęt gospodarza. ⁴⁴
Właściwość size dla punktu tmpfs	Blokuje przydzielanie nielimitowanej pojemności dla zrzutów tymczasowych skryptu w dyskach wirtualnych. ³⁸	Zapobiega podstępnej pochtanialności systemu przez ułudę działania z plikami na fałszywym nośniku blokowym.
--pids-limit=X	Bezwzględna i niepodważalna blokada rozmnażania potomków i limit klonowania procesów podrzędnych dla danego zasobu powłoki.	Tłumienie w zarodku zjawisk kaskadowych (Fork Bomb), zatrzymujące agenta na wczesnym etapie destrukcji procesorowej. ⁴¹

7. Model Context Protocol oraz Analiza Łańcucha Zaufania Systemów Agentowych

Zastąpienie hermetycznych architektur opartych o monolityczne skrypty nowym standardem komunikacyjnym Model Context Protocol (MCP) jest trendem absolutnie dominującym w rozwiązaniach roku 2026. Protokół ten stanowi ujednolicone spoivo zespajające niespójne dotąd modele językowe ze zróżnicowanymi, wbudowanymi zewnętrznymi danymi i potężnymi fizycznymi narzędziami interaktywnymi. Implementacja tego protokołu wiąże się nierozzerwalnie ze znacznym powiększeniem architektury podatności i otwarciem zupełnie nowej klasy niebezpieczeństw wymierzonych stricte na mechanizm zarządczy, sprowadzających się do procesów omijania, przejmowania kontekstu (Context Poisoning) czy zjawiska podszywania się modułów, potocznie ujmowanego jako kolizje i cieniowanie legalnych aplikacji wykonawczych

wirtualnego narzędziownika asystenta złośliwymi sobowtórami.⁴⁶

Dynamika Wielofazowych Przekroczeń Interfejsu i Brak Granic Logicznych

Niestabilność bezpieczeństwa oparta jest we wspomnianych systemach na problemie zarządzania zasadą najmniejszych przywilejów, co doprowadziło do fundamentalnego upadku logiki granic wykonawczych. Agent obdarzony powierzchownym zaufaniem realizuje niezależnie łańcuchowe sekwencje poszczególnych czynności wykonawczych o znikomym pojedynczym wpływie. Dysponując upoważnieniami odczytu własnej ścieżki i wolumenów pamięci masowej w warunkach odosobnionych, przywilejem komunikacji tekstowej i zewnętrznym nadawaniem statusów czy też ograniczonym dostępem do portów dla wywołań wewnątrz wirtualnej architektury piaskownicy, buduje sekwencje logiczne, stanowiące drogę kaskadową.⁹ Model może ulec wrogiemu wstrzyknięciu pochodzącemu pośrednio od informacji uzyskanej legalnie po przeskanowaniu zawartości i tym samym samodzielnie rozwinąć wektor odszukania i zbuforowania struktury parametrów certyfikujących węzłów. Dysponując pozyskanym kontekstem posłuży się w finalnej fazie legalnymi procedurami zapytania wychodzącego, przekierowując zawartość skradzioną do fałszywego portu i maskując swoje szkodliwe działanie wysłaniem autoryzowanego komunikatu rzekomego zakończenia powierzonego zadania diagnostycznego na kanał informacyjny programistów platformy w celu uspienia podejrzeń administratora powłoki.⁹ Rozwiązanie z 2026 roku nie uwzględnia analizy zachowania łańcuchów holistycznie.

Luki Łańcucha i Integracja Modeli: Katastrofa MCP

Przełomowe studia oparte na badaniach strukturalnych i wnikliwej kategoryzacji podatności architektonicznych z 2026 roku przeprowadzone za pomocą ewaluatora sklasyfikowanego jako MCPInspect i operujące na potężnej próbie badawczej obejmującej nieomal sześćdziesiąt siedem tysięcy instancji wirtualnych na publicznych listach, dowodzą gigantycznej słabości architektury zaufania na wyższym szczeblu. Wykazano druzgocącą tendencję wdrażania niecertyfikowanych repozytoriów, rzekomo powielających działanie narzędzi niezbędnych w rozwoju i integrowaniu platform piaskownicy, które to otwierały drogi wprowadzania złośliwej logiki, manipulowania parametrami schematu oraz przechwytywania logiki wykonawczej ze strony sztucznego pośrednika agentowego.⁴⁷ Skandalem deweloperskim i ogromnym wyłomem w architekturze wiary był potwierdzony przez analityków błąd znajdujący się bezpośrednio w uwiarygodnionej przez wiodącego dystrybutora implementacji służącej do modelowania integracji środowisk kodowych wewnątrz mechanizmu MCP. Ze względu na zaufanie programistów do rzekomo bezpiecznego źródła korporacyjnego, implementacja obarczona poważną podatnością, umożliwiającą wrogie połączenie środowiska i otwarcie interfejsu systemu, została skopiowana na setkach tysięcy lokalnych instalacji środowisk budujących, co zainicjowało dziedziczenie tej kardynalnej luki bezpieczeństwa i otwarcie infrastruktury badawczych piaskownic w korporacjach prywatnych.⁴⁹ Przykładem na namacalne i wysoce bezprawne nadużycia strukturalne oprogramowania inspekcyjnego w narzędziach dla administratorów do oceny interfejsu był zarejestrowany wektor operacyjny

CVE-2025-49596 stanowiący namacalny powód wygenerowania pełnego podłoża pod realizację działań hakerskich w obszarze RCE w narzędziach z uwagi na dramatyczny brak uwarunkowań walidacji wczytywanego pliku konfiguracyjnego środowiska.⁵⁰

8. Bezpieczeństwo Zależności i Wrogie

Oprogramowanie Generowane AI

Ogromna akceptacja społeczności i ślepe powierzenie zadań modelom w przestrzeni zawodowej dla systemów asystujących, potocznie określana ze względu na niezwykłą swobodę bytu terminu jako tak zwany "vibe coding", skutkuje zjawiskiem potężnego akumulowania się usterkowości logicznej w strukturach wypracowywanego podprogramu.⁵¹ Badania przeprowadzone w prestiżowych korporacjach udowodniły istnienie zjawiska o dramatycznych w skutkach proporcjach, wskazując, iż specjaliści w znacznej asyście struktur sztucznych tworzą w kodzie wielokrotnie częściej pojawiające się uszkodzenia struktury i zaniedbania na tle zabezpieczania procedur systemowych.⁵¹ Ogromna próba testowa polegająca na ocenie dziesiątek modeli ewaluacyjnych zaowocowała przerażającym werdyktem: niespełna połowa dostarczonych procedur operowała ze złamaniem najważniejszych doktryn bezpieczeństwa klasyfikowanych na listach OWASP, takich jak powszechne braki neutralizowania zapytań wektorowych bazy danych i defekty w budowie schematu szyfrującego platform, co w przypadku implementacji programowania przy użyciu struktury maszyny z obsługą platform Javy załamywało się dla przeszło ponad siedemdziesięciu procent podjętych analiz.⁵¹ Baza błędów Vibe Security Radar w roku 2026 zanotowała gwałtowną powtarzalność występowania oficjalnych ułomności i rejestrowania uchybień z ewidentnym obciążeniem zautomatyzowanych podprocesów deweloperskich AI odpowiedzialnością za ten stan.⁵¹

Złudzenie Biblioteczne i Katastrofalne Oszustwa Slopsquattingu

Wymagowana propozycja instalowania pakietów i operacji programistycznych (halucynowanie zależności), czyli uporczywe powoływanie się systemu asystującego na obiekty struktury zewnętrznej które nie figurują na platformach wydawniczych, to zjawisko destrukcyjne na niewyobrażalną do tej pory skalę; badania operacyjne i weryfikacje uniwersyteckie blisko miliona wygenerowanych podprocesów ujawniły obecność błędów tego twardego gatunku dla przeszło dwudziestu procent całości logiki.⁵¹ Specyficzne jest, że zachowanie sztucznych środowisk ma naturę skrajnie i wysoce precyzyjnie przewidywalną w oparciu o ich zasób leksykalny i zbitki pojęciowe wynikające ze splątań obiektowych wektorów maszyn wejściowych; około 38% tych nieprawidłowości bazuje na sklejeniu pojęciowym słów faktycznych narzędzi, a pojęcia absolutnie wyhalucynowane ze struktur nienazwanych stanowią połowę pozostałych pomyłek systemu generatywnego.⁵³

Agresywny wyścig cyberprzestępców oponuje zatem na nowym zjawisku, którego nazewnictwo skatalogowano jako wyrafinowany Slopsquatting.⁵⁴ Osoba atakująca środowiska wrogimi wektorami przejmując zarezerwowane przez błędne koncepcje nazw koncepcyjnych paczki i rezerwuje w odpowiednim repozytorium platform docelowych zasoby pod wymyśloną we wrogiej przestrzeni nazwę wyhalucynowanej abstrakcji algorytmu. Pakiet wysoce obciążony

wektorem oprogramowania instalującego niechciane tylne drzwi dostępu i wysysającego ukryte identyfikatory systemów bazowych zostaje opublikowany na rzecz środowiska zbrodniczego.⁵¹ W momencie nieświadomej autoryzacji na powłoce asystenta kodu wymuszającego uruchomienie sekwencji deweloperskiej w celu poszerzenia działania biblioteki dla procedury lub skryptu generatywnego obiekty te docierają w środowisko deweloperskie i uzyskują zwierzchność, nie zwracając uwagi systemu ochronnego z racji braku bliskiego pokrewieństwa ze sztucznymi zabezpieczeniami i autentycznymi plikami, w totalnej opozycji do starszych wektorów typo-graficznych i literówek stanowiących podstawy mechanizmów weryfikacji powszechnie stosowanych do wyłapywania oszustów.⁵³ Stanowi to niespotykany wektor infekcji łańcucha dostaw.

9. Podatności Komponentów Akceleracyjnych GPU Wewnątrz Piaskownicy

Oczekiwania stawiane środowiskom odizolowanym dedykowanym generatywnej analizie danych wymuszają od systemów przydzielania niezbędnych jednostek potężnych wieloprocessorowych akceleratorów wsparcia środowiska tensorowego poprzez przekierowanie węzła z instancji platformy gospodarza. Decyzja ta w sposób zasadniczy pomnaża powierzchowność atakowanych instancji o niezbadane i niezwykle słabe komponenty kontrolerów wirtualizacyjnych. Przełom 2025 roku uwypuklił krytyczne wadliwości powłok systemowych zarządzających przydzielaniem sprzętowym opublikowanych chociażby przez korporację NVIDIA (zjawisko ochrzczone symbolem NVIDIA Scape przy okazji luki w identyfikacji podsystemów CVE-2025-23266 o indeksie bazowym 9.0).²¹

Problematyka błędu dotyczyła naruszeń i modyfikacji zachowań w procesach implementacji inicjacyjnej i wykonywaniu rzekomo bezpiecznych punktów stykowych w obszarze montażu wirtualnego na specjalnym poziomie aktywacji (CDI) platform zunifikowanych do udostępniania bibliotek platform obciążeniowych dla kontenerów z rzekomymi systemami operacyjnymi o obniżonych przywilejach.²¹ Wygenerowanie kodu asystowanego uderzającego precyzyjnym argumentem w luki procedury inicjalizacyjnej powodowało bezpośrednie wyniesienie zasobów eskalujących autoryzację węzła dla piaskownicy omijając wysoce restrykcyjny status ograniczeń kontenerowych i zapewniając osobie nadużywającej procedury wolną i otwartą linię do dokonywania podstawnych inwigilacji macierzystego serwera hosta.⁵⁷

10. Konkluzje Architektoniczne i Bezwzględne

Rekomendacje dla Wdrażania Projektu Piaskownicy

Analiza całościowa uchybień na warstwie bezpieczeństwa, zdemontowanych mechanizmów opartych na języku interpretacyjnym JS oraz omijania systemów zapory i dekompozycji autoryzacji protokolowej w środowiskach opartych na autorskich strukturach we wczesnym 2026 roku nie pozostawia wątpliwości architektonicznych: budowa izolowanego stanowiska operacyjnego na naiwnych wytycznych, powszechnie uznawanych za bazowe i optymalne, stanowczo naraża infrastrukturę korporacyjną. Granica zaufania tradycyjnego obszaru programistycznego straciła swoją elementarną moc wiążącą, a piaskownica oparta wyłącznie o mechanizm warstwy podprogramów jest gwarantem zaistnienia natychmiastowego kryzysu i

zdalnego incydentu autoryzacji z poziomu agenta. Oczekiwania technologiczne narzucają na autora konieczność skreślenia znacznej części niepoprawnie skorelowanych warstw ochrony ze względu na niesatysfakcjonującą architekturę bezpieczeństwa. Na podstawie niniejszego audytu nakłada się poniższe wiążące rekomendacje systemowe:

1. **Odseparowanie Maszynowe – Obligatoryjna Podeszwa Wirtualizacji Opartej na KVM:** Wykorzystanie powszechnego rozwiązania platformowego opartego na warstwie wspólnego jądra (LXC) we wzorcach agentowych w pełni upoważnia zautomatyzowane byty AI do nadużywania dziur eksploatacyjnych środowiska gospodarczego. Architekturę wdrożenia kategorycznie zmusza się do porzucenia dotychczasowych kontenerów pozbawionych specjalnej podwarstwy silnika mikromaszyny typu MicroVM opartej na niezależnej separacji fizycznego wirtualizatora środowisk (na przykład z wykorzystaniem systemów dedykowanych Kata Containers i obudowujących infrastrukturę sieci typu Firecracker).³ Dla środowisk analitycznych niereagujących w systemy powłok i wykonujących skrypty statystyczne w sposób odseparowany wyznacza się i wysoce aprobuje nowatorskie paradygmaty wykorzystujące do wykonania zamkniętego technologię WebAssembly z prekompilowanymi środowiskami dla kodu asystowanego przez modele języka Python (Pyodide), ucinając całkowitą możliwość użycia protokołów wyjściowych.⁴
2. **Ograniczenia Przemysłowe Komunikacji Zewnętrznej (Bezwzględna Polityka Egress):** Implementacja systemowa skryptów nakładających granice komunikacyjne dla ruchu zorientowanego na chmury informacyjne wymaga stosowania zapor opartych o zaawansowane skanowanie ze zrozumieniem protokołów warstwy wyższej i z pełną analizą strumieni (eliminujące usterkowość architektoniczną wyrafinowanych cięć fragmentaryzacji i ataków Slicing eBPF na zaporach sprzętowych).³² Ruch wykorzystujący technologie szyfrowane na styku z publicznymi mechanizmami serwerów w warstwie tunelowanej (DNS over HTTPS) musi podlegać regułom wyłapywania z warstwy agentowej; w obiekcie stosowanym przez model AI do połączeń nakazuje się restrykcyjne filtrowanie operacyjne według ścisłych list oznaczonych na zaporach typu whitelist.³ Niedopuszczalne są instalacje węzłowe z naruszonym zaufaniem routowania klastrowego platformy Docker dopuszczające omijanie wewnętrznej zapory firewalla i pozwalające na nasłuch otwarty dla kontenerów o pominiętej zmiennej weryfikacyjnej (NO_PROXY) dla zasobów lokalnych bez potwierdzenia tożsamości żądającego modułu.³⁴
3. **Twarda Izolacja i Wyłącznieść Dostępowa do Narzędzi API OS-Level:** Weryfikacja kodów wykonawczych we wbudowanej infrastrukturze platformy JS (takiej jak wyeksploatowana i nieoptymalna do dalszego łatania wersja vm2 używana nagminnie we frameworkach generatywnych do powoływania asystentów LLM) stanowi wyłącznie iluzoryczny porządek pracy obiektu.² Zmiana polityki nadzorowania narzuca stanowczy rygor nakładania reguł dostępu z najniższej możliwej do uchwycenia przestrzeni profili bezpieczeństwa samego jądra hosta (Seatbelt, odpowiednie konfiguracje blokujące Seccomp i ograniczone przywileje do warstw systemowych narzędzia MCP). Narzędziom zablokuje się fizyczny przywilej wykonywania wewnątrz obszarów roboczych polegający

na modyfikacji reguł odczytów plików konfiguracyjnych lub odnajdywaniu i eksponowaniu wrażliwych zmiennych procesowych dla kluczy systemowych.³

4. **Bariery i Cenzura Działalności Zasobów Infrastrukturalnych:** Piaskownica AI pozostawiona samej sobie operuje bezwzględny modelem statystyki zepsucia; system odmawia w krótkim wymiarze prawidłowych zadań dla klastra ze względu na uśmiercenie węzła procedurą jądra pozbawionego pojemności dla obciążenia sterty tymczasowych ramdysków.³⁶ Koniecznym punktem architektonicznym staje się zaawansowane sztywnienie rozmiarów dyskowych do wirtualnych zasobów alokacyjnych plikowej pamięci operacyjnej (zmienna wolumenu tmpfs ograniczana przez flagę systemową Dockera o parametrze precyzyjnie kalkulującym przydział ramów dla kontenera). Bezwzględnie aplikowane są mechanizmy kontrolera przestrzeni procesów zablokowane dla mnożenia bytu w ramach rekursywnych iteracji i uśmiercające skrajnie nieoptymalny proces zanim maszyna hosta wejdzie w globalną katastrofę braku możliwości działania pod zjawiskiem bomby klonującej dla wezwania `--pids-limit`.³⁸
5. **Polityka Destrukcji i Zarządzania Bezpieństwem Cyklem Istnienia (Stateless Ephemerality):** Przez wzgląd na nieszczelność wyciekających struktur oprogramowania łańcucha dostaw pod przykrywką asystowanych zależności (halucynowane instalatory slopsquattingu kradnące informacje tuż po pobraniu) przewencyjnie stosuje się reżim niszczenia środowiska dla każdego zakończonego i ocenionego w piaskownicy etapu przetwarzania wiedzy logiki agenta sztucznej.³ Zabronione jest używanie trwałych obszarów podmontowanej trwałej dominacji systemu plików i wymusza się nieustanną kreację wirtualizatorów maszyny wejściowej od początku, blokując skompromitowanym modułom zaszczerpiętym na warstwę piaskownicy perspektywę osadzania w trybach podprogramu ukrytych furt dla napastników zewnętrznych analizujących platformę na dłuższą metę deweloperską. Powłoki generowane przez złośliwy kontekst przestają egzystować szybciej, niż pozyskać zdołają wystarczające przywileje dominujące do wdarcia się w gniazda autoryzacji demonów środowiska macierzystego korporacji opartych na skrajnie wrażliwych usługach autoryzacji Dockera ze spuchniętymi ramkami komunikacyjnymi warstwy wyższej HTTP.³

Cytowane prace

1. OWASP GenAI Exploit Round-up Report Q1 2026, otwierano: czerwca 7, 2026, <https://genai.owasp.org/2026/04/14/owasp-genai-exploit-round-up-report-q1-2026/>
2. vm2 Sandbox Escape | AI Agent RCE Risk and IOCs | Kodem, otwierano: czerwca 7, 2026, <https://www.kodemsecurity.com/resources/vm2-sandbox-escape-vulnerabilities-the-2026-cve-wave-turning-ai-agents-into-host-rce-vectors>
3. Practical Security Guidance for Sandboxing Agentic Workflows and ..., otwierano: czerwca 7, 2026, <https://developer.nvidia.com/blog/practical-security-guidance-for-sandboxing-a-gentic-workflows-and-managing-execution-risk/>

4. Building a Secure Code Sandbox for LLMs with WebAssembly | by ..., otwierano: czerwca 7, 2026,
<https://medium.com/collaborne-engineering/building-a-secure-code-sandbox-for-llms-with-webassembly-bdd91a835f23>
5. What's the best code execution sandbox for AI agents in 2026 ..., otwierano: czerwca 7, 2026,
<https://northflank.com/blog/best-code-execution-sandbox-for-ai-agents>
6. OpenClaw Security Analysis: Critical Vulnerabilities in Docker Socket Exposure, otwierano: czerwca 7, 2026,
<https://productos-ai.com/blog/openclaw-security-analysis>
7. CVE-2026-27002: OpenClaw Privilege Escalation Vulnerability - SentinelOne, otwierano: czerwca 7, 2026,
<https://www.sentinelone.com/vulnerability-database/cve-2026-27002/>
8. A Security Analysis of the OpenClaw AI Agent Framework - arXiv, otwierano: czerwca 7, 2026, <https://arxiv.org/html/2603.27517v3>
9. AI Agent Security Risks: When Autonomous Systems Fail - Redfox Cybersecurity, otwierano: czerwca 7, 2026,
<https://www.redfoxsec.com/blog/ai-agent-security-risks-what-autonomous-systems-can-do-when-they-go-wrong>
10. Why sandboxing coding agents is harder than you think - Martin Alderson, otwierano: czerwca 7, 2026,
<https://martinalderson.com/posts/why-sandboxing-coding-agents-is-harder-than-you-think/>
11. Firecracker vs gVisor: Which isolation technology should you use? | Blog - Northflank, otwierano: czerwca 7, 2026,
<https://northflank.com/blog/firecracker-vs-gvisor>
12. Security Model - gVisor, otwierano: czerwca 7, 2026,
https://gvisor.dev/docs/architecture_guide/security/
13. gVisor vs Kata Containers vs Firecracker MicroVMs on VPS in 2025: Isolation - Onidel, otwierano: czerwca 7, 2026,
<https://onidel.com/blog/gvisor-kata-firecracker-2025>
14. Firecracker, gVisor, Containers, and WebAssembly - Comparing Isolation Technologies for AI Agents - SoftwareSeni, otwierano: czerwca 7, 2026,
<https://www.softwareseni.com/firecracker-gvisor-containers-and-webassembly-comparing-isolation-technologies-for-ai-agents/>
15. Security and Vulnerability Reporting - gVisor, otwierano: czerwca 7, 2026,
<https://gvisor.dev/security/>
16. Security and Vulnerability Reporting - google/gvisor - GitHub, otwierano: czerwca 7, 2026, <https://github.com/google/gvisor/security>
17. Security bulletins | Google Kubernetes Engine (GKE), otwierano: czerwca 7, 2026,
<https://docs.cloud.google.com/kubernetes-engine/security-bulletins>
18. Kata, gVisor, or Firecracker? Container Isolation Guide - Edera, otwierano: czerwca 7, 2026,
<https://edera.dev/stories/kata-vs-firecracker-vs-gvisor-isolation-compared>
19. Critical runC Vulnerabilities Allow Container Escape in Docker, Kubernetes |

- Quantum Safe News Center - Gopher Security, otwierano: czerwca 7, 2026,
<https://www.gopher.security/news/critical-runc-vulnerabilities-allow-container-escape-in-docker-kubernetes>
20. CVE-2025-31133, CVE-2025-52565, CVE-2025-52881 - runc container issues - AWS - Amazon.com, otwierano: czerwca 7, 2026,
<https://aws.amazon.com/security/security-bulletins/rss/aws-2025-024/>
 21. Container Escape Vulnerabilities: AI Agent Security for 2026 | Blaxel Blog, otwierano: czerwca 7, 2026, <https://blaxel.ai/blog/container-escape>
 22. New runc vulnerabilities allow container escape: CVE-2025-31133 ..., otwierano: czerwca 7, 2026,
<https://www.sysdig.com/blog/runc-container-escape-vulnerabilities>
 23. runc container breakout vulnerabilities: A technical overview | CNCF, otwierano: czerwca 7, 2026,
<https://www.cncf.io/blog/2025/11/28/runc-container-breakout-vulnerabilities-a-technical-overview/>
 24. New runC Vulnerabilities Expose Docker and Kubernetes to Container Escape Attacks, otwierano: czerwca 7, 2026,
<https://orca.security/resources/blog/new-runc-vulnerabilities-allow-container-escape/>
 25. Docker Security Flaw CVE-2026-34040: Patch Now | Cyera Research, otwierano: czerwca 7, 2026,
<https://www.cyera.com/research/one-megabyte-to-root-how-a-size-check-broke-dockers-last-line-of-defense>
 26. CVE-2026-31431 Docker Container Escape Vulnerabilities in 2026 - Sesame Disk, otwierano: czerwca 7, 2026,
<https://sesamedisk.com/container-escape-vulnerabilities-2026/>
 27. How to implement DNS requests filtering for my docker containers and stacks - Reddit, otwierano: czerwca 7, 2026,
https://www.reddit.com/r/docker/comments/1dayn81/how_to_implement_dns_requests_filtering_for_my/
 28. docker egress filtering / firewall - Stack Overflow, otwierano: czerwca 7, 2026,
<https://stackoverflow.com/questions/45100053/docker-egress-filtering-firewall>
 29. More egress filtering bypasses in harden-runner - devansh, otwierano: czerwca 7, 2026, <https://devansh.bearblog.dev/harden-runner-bypass/>
 30. CargoWall eBPF Firewall · Actions · GitHub Marketplace, otwierano: czerwca 7, 2026, <https://github.com/marketplace/actions/cargowall-ebpf-firewall>
 31. eBPF Applications Landscape, otwierano: czerwca 7, 2026,
<https://ebpf.io/applications/>
 32. Breaking the “Unbreakable” eBPF Firewall: A Tale of Statelessness and Slices | UofTCTF 2026 - Medium, otwierano: czerwca 7, 2026,
<https://medium.com/@0xs3l/breaking-the-unbreakable-ebpf-firewall-a-tale-of-statelessness-and-slices-uoftctf-2026-5c0a028d6866>
 33. lawrencegripper/ebpf-cgroup-firewall - GitHub, otwierano: czerwca 7, 2026,
<https://github.com/lawrencegripper/ebpf-cgroup-firewall>
 34. Implementing Network Egress Control in Docker Swarm | BlockQueue Systems

- Limited, otwierano: czerwca 7, 2026,
<https://blockqueue.io/blog/2026-02-15-docker-swarm-network-egress-guide>
35. Docker Network bypasses Firewall, no option to disable · Issue #22054 - GitHub, otwierano: czerwca 7, 2026,
https://github.com/moby/moby/issues/22054?timeline_page=1
 36. Container memory usage keeps increasing until OOM : r/kubernetes - Reddit, otwierano: czerwca 7, 2026,
https://www.reddit.com/r/kubernetes/comments/1alq261/container_memory_usage_keeps_increasing_until_oom/
 37. DevOps Scenario #11: Why Your Docker Container Exceeds Memory Limits-Deep Dive Into cgroups, Namespaces & The OOM Killer - Medium, otwierano: czerwca 7, 2026,
<https://medium.com/@mdmarjanrafi/devops-scenario-11-why-your-docker-container-exceeds-memory-limits-deep-dive-into-cgroups-7c4930633d2c>
 38. How to Use Docker Compose tmpfs Configuration - OneUptime, otwierano: czerwca 7, 2026,
<https://oneuptime.com/blog/post/2026-02-08-how-to-use-docker-compose-tmpfs-configuration/view>
 39. Resource constraints - Docker Docs, otwierano: czerwca 7, 2026,
https://docs.docker.com/engine/containers/resource_constraints/
 40. Docker: How to increase the size of tmpfs volumes? - Stack Overflow, otwierano: czerwca 7, 2026,
<https://stackoverflow.com/questions/54911335/docker-how-to-increase-the-size-of-tmpfs-volumes>
 41. How to Use Podman as a Sandbox for Untrusted Code - OneUptime, otwierano: czerwca 7, 2026,
<https://oneuptime.com/blog/post/2026-03-18-use-podman-sandbox-untrusted-code/view>
 42. What The Boooomb? — The Fork Bomb Explained | by Joseph Mwangi - Medium, otwierano: czerwca 7, 2026,
https://medium.com/@Joseph_Mwangi/what-the-boooomb-the-fork-bomb-explained-48785bcc3a78
 43. Support `--pids-limit` docker run` flag · Issue #354 · aws/containers-roadmap - GitHub, otwierano: czerwca 7, 2026,
<https://github.com/aws/containers-roadmap/issues/354>
 44. Ship It: Docker Security, Scout & What Comes Next | Day 7 | Kubesimplify, otwierano: czerwca 7, 2026,
<https://blog.kubesimplify.com/day-7-ship-it-and-what-comes-next>
 45. Docker Engine 1.10 Security Improvements, otwierano: czerwca 7, 2026,
<https://www.docker.com/blog/docker-engine-1-10-security/>
 46. Model Context Protocol: Security Risks & Mitigations - SOC Prime, otwierano: czerwca 7, 2026,
<https://socprime.com/blog/mcp-security-risks-and-mitigations/>
 47. 11 Emerging AI Security Risks with MCP (Model Context Protocol) - Checkmarx, otwierano: czerwca 7, 2026,
<https://checkmarx.com/zero-post/11-emerging-ai-security-risks-with-mcp-mode>

[l-context-protocol/](#)

48. [2510.16558] A First Look at the Security Issues in the Model Context Protocol Ecosystem, otwierano: czerwca 7, 2026, <https://arxiv.org/abs/2510.16558>
49. MCP (Model Context Protocol) is moving fast — and so are the attackers., otwierano: czerwca 7, 2026, https://www.reddit.com/r/cybersecurity/comments/1s5vvhy/mcp_model_context_protocol_is_moving_fast_and_so/
50. Model Context Protocol (MCP): Security Design Considerations for AI-Driven Automation, otwierano: czerwca 7, 2026, https://www.nsa.gov/Portals/75/documents/Cybersecurity/CSI_MCP_SECURITY.pdf?ver=bmqiSbNQLP6Z_GiWtRt6bg%3D%3D
51. Vibe Coding's Security Debt: The AI-Generated CVE Surge – Lab ..., otwierano: czerwca 7, 2026, <https://labs.cloudsecurityalliance.org/research/csa-research-note-ai-generated-code-vulnerability-surge-2026/>
52. AI-Driven Hallucinations in Cyber Supply Chain Lead to New Threat: Slopsquatting, otwierano: czerwca 7, 2026, <https://www.capttechu.edu/blog/ai-driven-threats-in-software-supply-chains>
53. Slopsquatting: The AI Package Hallucination Attack Already Happening - Aikido Security, otwierano: czerwca 7, 2026, <https://www.aikido.dev/blog/slopsquatting-ai-package-hallucination-attacks>
54. Slopsquatting: When AI Agents Hallucinate Malicious Packages | Trend Micro (TR), otwierano: czerwca 7, 2026, <https://www.trendmicro.com/vinfo/tr/security/news/cybercrime-and-digital-threats/slopsquatting-when-ai-agents-hallucinate-malicious-packages>
55. The Hallucinated Package Attack: Slopsquatting Explained - Mend.io, otwierano: czerwca 7, 2026, <https://www.mend.io/blog/the-hallucinated-package-attack-slopsquatting/>
56. CVE-2025-23266 - Amazon Linux Security Center, otwierano: czerwca 7, 2026, <https://explore.alas.aws.amazon.com/CVE-2025-23266.html>
57. NVIDIA Container Toolkit for all platforms contains an Untrusted Search Path · CVE-2025-23266 · GitHub Advisory Database, otwierano: czerwca 7, 2026, <https://github.com/advisories/GHSA-vmg3-7v43-9g23>
58. CVE-2025-23266 - Red Hat Customer Portal, otwierano: czerwca 7, 2026, <https://access.redhat.com/security/cve/cve-2025-23266>
59. CVE-2025-23266 Detail - NVD, otwierano: czerwca 7, 2026, <https://nvd.nist.gov/vuln/detail/cve-2025-23266>